

## AI Engineering Cohort

# Week 3 – Day 2

Skip Connections, Layer Norm, Multi-Head Attention, Masking & Training vs Inference

March 15, 2026 | Instructor: Gaurav

## 1. Attention Recap — The Numbers

Quick reference: the full 'There is no tomorrow' attention calculation from the official lecture notes.

| Word       | Resultant Embedding (word + position) |
|------------|---------------------------------------|
| there_1    | [1.25, 1.0, 0.5]                      |
| is_2       | [0.0, 1.25, 1.5]                      |
| no_3       | [1.5, 0.5, 1.0]                       |
| tomorrow_4 | [1.25, 1.1, 0.0]                      |

Attention scores for key 'tomorrow\_4': there=2.84, is=1.26, no=2.33, tomorrow=3.09. Scaled by  $\sqrt{3}=1.73$ : [1.64, 0.72, 1.34, 1.78]. After Softmax: [0.30, 0.12, 0.23, 0.35]. Contextualised output = [1.16, 0.94, 0.52].



Normalisation is applied immediately after attention. Official notes worked example: there\_1=[1.43,0.87,0.78] → mean=1.027, variance calculated → normalised=[1.34,0.82,0.73]. LayerNorm makes the optimisation landscape smoother. RMSNorm is 10-60% faster and is what production models use today.

## 2. Skip Connections (Residual Connections)

Skip connections are one of the most important architectural innovations in deep learning. Without them, training transformers (and deep networks in general) is nearly impossible due to vanishing gradients.

### 2.1 The Problem They Solve

As gradients flow backwards through many layers during backpropagation, they can shrink exponentially — this is the vanishing gradient problem. By the time gradients reach the early layers, they are so small the weights barely update. Training becomes unstable and slow.



Highway analogy from class: You need to get from point A to point B. The route is A →

Tunnel → Potholes → Bridge → Tunnel → B. If one tunnel is blocked, you are stuck. A skip connection is like a guaranteed highway that runs parallel — even if some sub-layers learn nothing, the original information can always flow through directly.

## 2.2 How They Work — The Formula

Without a skip connection:  $\text{output} = \text{Layer}(\text{input})$ . If the layer learns garbage, the output is garbage — and the original input is lost.

With a skip connection (residual connection):  $\text{output} = \text{input} + \text{Layer}(\text{input})$ . If the layer learns nothing useful,  $\text{Layer}(\text{input}) \approx 0$ , and  $\text{output} \approx \text{input}$ . The original information is preserved.

```
# Without skip connection:
output = layer(input)    # if layer fails → garbage output

# With skip connection (residual):
output = input + layer(input)    # if layer fails → output = input (info preserved)
```

| Scenario                                             | Without Skip                          | With Skip                                                                                     |
|------------------------------------------------------|---------------------------------------|-----------------------------------------------------------------------------------------------|
| Layer learns correctly                               | Good output                           | Good output (input + improvement)                                                             |
| Layer learns nothing ( $\text{Layer}(x) \approx 0$ ) | Garbage output — original info lost   | $\text{output} = \text{input} + 0 = \text{input}$ — preserved                                 |
| Gradient flow through 12 layers                      | Gradient shrinks to near 0 by layer 3 | Gradient always has a direct path: $\partial H / \partial x = 1 + \partial F(x) / \partial x$ |

The gradient formula shows why this works mathematically:

$$\partial H / \partial x = \partial (x + F(x)) / \partial x = 1 + \partial F(x) / \partial x$$

The '1' term guarantees a gradient of at least 1 always flows back — no matter how many layers deep. This is what allows transformers to be trained with 12, 32, or 100+ layers.

### TA Note / Class Clarification

The original transformer architecture uses:  $\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$ . This is Post-Norm (normalise after adding skip). Modern architectures (LLaMA, GPT-4 etc.) use Pre-Norm:  $\text{output} = x + \text{Sublayer}(\text{LayerNorm}(x))$ . Pre-Norm is more stable for very deep models.

## 3. Layer Normalisation

After the attention block (with its skip connection), and again after the FFN (with its skip connection), a normalisation step is applied. This keeps vector values at a consistent scale throughout the network.

### 3.1 The Problem

As signals pass through layers, their scale (mean and variance) changes. After layer 1, mean might be 0.5 and variance 1.2. After layer 2: mean 2.3, variance 5.7. After layer 3: mean 10.1, variance 50.2. The values explode. Some neurons update aggressively, others barely move. Training becomes chaotic.

### 3.2 Layer Norm Formula

Layer Norm rescales each vector so it has mean=0 and variance=1, then applies learnable scale ( $\gamma$ ) and shift ( $\beta$ ) parameters:

$$\text{LN}(x) = \gamma \times (x - \text{mean}(x)) / \sqrt{\text{variance}(x) + \epsilon} + \beta$$

Where:

$\text{mean}(x)$  = average of all dimensions in the vector

$\text{variance}(x)$  = spread of values around the mean

$\epsilon$  = small constant (e.g.  $1e-5$ ) to prevent division by zero

$\gamma, \beta$  = learnable parameters (scale and shift) – trained by backprop

### 3.3 Worked Example (from handwritten notes)

Given vector  $x = [1.0, 2.0, 3.0, 4.0, 5.0]$ :

$$\text{mean}(x) = (1+2+3+4+5)/5 = 3.0$$

$$\begin{aligned} \text{variance}(x) &= [(1-3)^2 + (2-3)^2 + (3-3)^2 + (4-3)^2 + (5-3)^2] / 5 \\ &= [4 + 1 + 0 + 1 + 4] / 5 = 2.0 \end{aligned}$$

$$\begin{aligned} x_{\text{norm}} &= (x - 3.0) / \sqrt{2.0 + 0.00001} \\ &= ([1, 2, 3, 4, 5] - 3) / \sqrt{2.0} \\ &= [-2, -1, 0, 1, 2] / 1.414 \\ &= [-1.414, -0.707, 0, 0.707, 1.414] \end{aligned}$$

With initial  $\gamma=[1,1,1,1,1]$  and  $\beta=[0,0,0,0,0]$ :

$$\text{output} = \gamma \times x_{\text{norm}} + \beta = [-1.414, -0.707, 0, 0.707, 1.414]$$

Now mean=0, variance=1. As training continues,  $\gamma$  and  $\beta$  are updated by backpropagation to find the optimal scale and shift for each layer.

| Type       | What It Normalises                          | Used In                           | Notes                                                   |
|------------|---------------------------------------------|-----------------------------------|---------------------------------------------------------|
| Batch Norm | Across samples in a batch (per feature)     | CNNs, early networks              | Bad for variable-length sequences; batch-size dependent |
| Layer Norm | Across all features in one sample           | Original transformer, BERT, GPT-2 | Better for sequences; batch-size independent            |
| RMSNorm    | Root mean square only (no mean subtraction) | LLaMA, GPT-4, modern LLMs         | 10-60% faster than LayerNorm; current standard          |

### 3.4 Pre-Norm vs Post-Norm

| Variant              | Formula                                                    | Used In                        | Notes                                            |
|----------------------|------------------------------------------------------------|--------------------------------|--------------------------------------------------|
| Post-Norm (original) | $\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$ | Original 2017 transformer      | Can be unstable for very deep models             |
| Pre-Norm (modern)    | $\text{output} = x + \text{Sublayer}(\text{LayerNorm}(x))$ | LLaMA, GPT-4, most modern LLMs | More stable; better gradient flow at large depth |

```
# Post-Norm (original transformer):
class PostNormLayer(nn.Module):
    def forward(self, x):
        attn_out = self.attention(x)
        x = self.norm1(x + attn_out)           # add then norm
        ffn_out = self.ffn(x)
        x = self.norm2(x + ffn_out)           # add then norm
        return x

# Pre-Norm (modern):
class PreNormLayer(nn.Module):
    def forward(self, x):
        attn_out = self.attention(self.norm1(x)) # norm then sub-layer
        x = x + attn_out                          # add skip
        ffn_out = self.ffn(self.norm2(x))         # norm then sub-layer
        x = x + ffn_out                          # add skip
        return x
```

## 4. Multi-Head Attention

Single-head attention computes one set of Q, K, V projections and finds one type of relationship between tokens. Multi-head attention runs the attention mechanism  $H$  times in parallel, each with different learned weight matrices — allowing the model to simultaneously attend to multiple different aspects of the input.

### 4.1 Why Multiple Heads?

Consider being evaluated in a job interview. One interviewer asking all questions can only focus on one aspect at a time — they might ask about coding, then knowledge, then teamwork, but sequentially. What if the technical interviewer, the knowledge interviewer, and the HR interviewer could all ask questions simultaneously?

| Head | What It Might Learn | Example for sentence: 'The cat sat on the mat' |
|------|---------------------|------------------------------------------------|
|------|---------------------|------------------------------------------------|

|                         |                                        |                                            |
|-------------------------|----------------------------------------|--------------------------------------------|
| Head 1<br>(Syntactic)   | Subject-verb-object relationships      | 'cat' → 'sat': who is doing the action     |
| Head 2<br>(Semantic)    | Semantic similarity / meaning clusters | 'mat' → 'sat': where the action happened   |
| Head 3<br>(Coreference) | Pronoun/reference resolution           | 'it' → 'cat': what 'it' refers to          |
| Head 4<br>(Positional)  | Nearby word relationships              | Adjacent word patterns and n-grams         |
| Head 5-12<br>(Emergent) | Patterns learned purely from data      | Syntactic roles, discourse structure, etc. |



The model decides what each head learns — we do not program the roles above. These patterns are discovered by the model through training. Research (e.g., Voita et al. 2019) has found that different heads do specialise in different linguistic functions, but the assignment is emergent, not designed.

## 4.2 Multi-Head Architecture

Each head  $h$  has its own weight matrices:  $WQ_h$ ,  $WK_h$ ,  $WV_h$ . The input  $X$  is projected into a different  $Q$ ,  $K$ ,  $V$  for each head. Each head computes attention independently. All head outputs are concatenated and projected through a final weight matrix  $W_O$ .

For head  $h$ :

$$Q_h = X \times WQ_h \quad (d_{\text{model}} \times d_k)$$

$$K_h = X \times WK_h \quad (d_{\text{model}} \times d_k)$$

$$V_h = X \times WV_h \quad (d_{\text{model}} \times d_v)$$

$$\text{head}_h = \text{Attention}(Q_h, K_h, V_h) = \text{Softmax}(Q_h \times K_h^T / \sqrt{d_k}) \times V_h$$

Final output:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_H) \times W_O$$

$W_O$  shape:  $(H \times d_v) \times d_{\text{model}}$  — projects concatenated heads back to  $d_{\text{model}}$

Key dimension relationship:  $d_k = d_v = d_{\text{model}} / H$ . So each head works in a smaller dimensional space, keeping the total computation approximately equal to single-head attention.

| Model                                   | $d_{\text{model}}$ | Heads ( $H$ ) | $d_k$ per head | Total MHA params                                                                                   |
|-----------------------------------------|--------------------|---------------|----------------|----------------------------------------------------------------------------------------------------|
| Original Transformer                    | 512                | 8             | 64             | ~1M                                                                                                |
| GPT-2 (small)                           | 768                | 12            | 64             | ~3M                                                                                                |
| GPT-3                                   | 12288              | 96            | 128            | ~1.5B                                                                                              |
| From handwritten notes: 1 encoder layer | 512                | 8             | 64             | MHA:<br>$3 \times (512 \times 512) = 786,432$ +<br>$W_O: 512 \times 512 = 262,144$<br>$\approx 1M$ |

**Quick Check****Q: If  $d_{\text{model}}=512$  and  $H=8$  heads, what is  $d_k$  for each head? Why?**

A:  $d_k = 512/8 = 64$ . Each head works in a 64-dimensional subspace. This keeps total computation roughly equal to single-head attention with  $d_k=512$ , since  $8 \text{ heads} \times 64 \text{ dimensions} = 512 \text{ total}$ . The intuition: rather than one 512-dimensional attention, you get eight 64-dimensional attentions that specialise in different relationships.

## 5. Three Types of Attention

The transformer architecture uses three distinct attention mechanisms, each with different source sequences for Q, K, V and different masking behaviour.

| Type                  | Where                 | Q Source                              | K, V Source                           | Masking                                      | Directionality                                         |
|-----------------------|-----------------------|---------------------------------------|---------------------------------------|----------------------------------------------|--------------------------------------------------------|
| Self-Attention        | Encoder               | Same sequence                         | Same sequence                         | None                                         | Bi-directional — every token can see every other token |
| Masked Self-Attention | Decoder (sub-layer 1) | Same sequence (decoder output so far) | Same sequence (decoder output so far) | Causal mask — future tokens set to $-\infty$ | Uni-directional — can only see previous tokens         |
| Cross-Attention       | Decoder (sub-layer 2) | Decoder sequence                      | Encoder output (K and V from encoder) | None                                         | Decoder attends to entire encoder output               |

### 5.1 Self-Attention (Encoder) — Visual

Sentence: 'The cat sat'. Attention matrix (3×3) — every token can attend to every other token:

|     | The | cat | sat |               |
|-----|-----|-----|-----|---------------|
| The | [ • | •   | • ] | ← can see all |
| cat | [ • | •   | • ] | ← can see all |
| sat | [ • | •   | • ] | ← can see all |

All cells filled = bi-directional = full context

### 5.2 Masked Self-Attention (Decoder) — Visual

Generating French: 'J' aime le'. Each token can only attend to itself and previous tokens (lower triangular):

|      | J'        | aime | le |                        |
|------|-----------|------|----|------------------------|
| J'   | [ • × × ] |      |    | → only sees itself     |
| aime | [ • • × ] |      |    | → sees previous tokens |
| le   | [ • • • ] |      |    | → sees all previous    |

× = masked (set to  $-\infty$  before softmax → becomes 0% after softmax)

• = unmasked (normal attention score)

Masking is implemented by a lower triangular matrix  $M$  where  $M[i,j] = 0$  if  $j \leq i$  (allowed),  $M[i,j] = -\infty$  if  $j > i$  (masked). The masked attention scores:  $Z = \text{Softmax}((QK^T + M) / \sqrt{dk}) \times V$ .

### 5.3 Cross-Attention (Decoder) — Visual

Translating English 'I love you' → French. Decoder is generating 'J' aime le'. Cross-attention matrix is 3×3 (3 French words × 3 English words):

|      | I         | love | you |                              |
|------|-----------|------|-----|------------------------------|
| J'   | [ • • • ] |      |     | → queries all encoder tokens |
| aime | [ • • • ] |      |     | → queries all encoder tokens |
| le   | [ • • • ] |      |     | → queries all encoder tokens |

Q: from decoder (French words being generated)

K, V: from encoder output (English words – already fully processed)

Major difference: WHERE Q, K, V come from + WHETHER masked



The three-attention structure means the decoder has three jobs: (1) Masked Self-Attention — understand what I have generated so far. (2) Cross-Attention — understand what the input (source) says. (3) FFN — process and transform. This is why the decoder has 3 sub-layers while the encoder has only 2.



#### TA Note / Class Clarification

Misconception: 'Masking only happens in the decoder.' Actually, masking also happens in the encoder when sentences are padded. Because all sequences in a batch must be the same length, short sentences are padded with dummy tokens. The model should not attend to padding — so those positions are also masked. The encoder uses `max_seq_length` (e.g., 512) and pads shorter sentences.

## 6. Training vs Inference — Teacher Forcing and Autoregressive Decoding

The transformer behaves differently during training and inference. Understanding this distinction is critical for understanding why masking exists and how the model actually generates text.

## 6.1 Training — Teacher Forcing

During training, the model sees the entire correct output sequence at once. It does not use its own (potentially wrong) predictions as the next input — it uses the ground truth. This technique is called Teacher Forcing.

Example: training on 'There is no tomorrow Rocky.'

- Input to encoder: 'There is no tomorrow'
- Target (shifted right): 'is no tomorrow Rocky'
- At each position, the model predicts the next token while masked self-attention prevents it from seeing future positions
- All 4 positions are processed in parallel — this is why training is fast
- Loss computed for all positions simultaneously



Teacher Forcing is why training is parallelisable. The model predicts all tokens simultaneously (with masking to simulate sequential generation). The mask makes the parallel computation behave as if the model generated tokens one by one. This is the key trick that makes transformer training on GPU so efficient.

## 6.2 Inference — Causal Autoregressive Decoding

During inference, the model generates one token at a time. Each generated token is fed back as input to generate the next. This is called autoregressive generation (or causal decoding).

```
Step 1: Input=[START] → Model predicts 'There'
Step 2: Input=[START, There] → Model predicts 'is'
Step 3: Input=[START, There, is] → Model predicts 'no'
Step 4: Input=[START, There, is, no] → Model predicts 'tomorrow'
Step 5: Input=[START, There, is, no, tomorrow] → Model predicts 'Rocky'
Step 6: Input=[START, There, is, no, tomorrow, Rocky] → Predicts [END]
```

Because inference is sequential, it is slower than training. KV-caching (not covered in this class) can optimise this by caching previously computed key and value vectors.

|                  | Training (Teacher Forcing)          | Inference (Autoregressive)                      |
|------------------|-------------------------------------|-------------------------------------------------|
| Processing       | Parallel — all tokens at once       | Sequential — one token at a time                |
| Input to decoder | Ground truth tokens (shifted)       | Own previous predictions                        |
| Speed            | Fast — GPU parallelism              | Slower — sequential dependency                  |
| Masking          | Essential — prevents future-peeking | Naturally not needed (future doesn't exist yet) |
| Also called      | Teacher Forcing                     | Causal Autoregressive Decoding                  |



The reason masking is needed at all: training is parallel (all positions processed at once), so without masking, the model could 'cheat' by looking at future tokens. Masking simulates the sequential nature of inference during training. It makes the parallel training

behave identically to what the sequential inference will do.

## 7. Parameter Counts — How Models Reach Billions

Your handwritten notes worked through the exact parameter counts for the original transformer (6 encoder + 6 decoder layers,  $d_{\text{model}}=512$ ,  $H=8$  heads,  $d_{\text{ff}}=2048$ ). This is a crucial exercise for understanding why model sizes are in billions.

### 7.1 One Encoder Layer

Note:  $d_k = d_{\text{model}} / H = 512 / 8 = 64$  for this model.

Multi-Head Attention (MHA):

$W_Q, W_K, W_V: 3 \times (512 \times 512) = 3 \times 262,144 = 786,432$

$W_O$  (output projection):  $512 \times 512 = 262,144$

MHA total:  $\approx 1,048,576 \approx 1\text{M}$  params

Feed Forward Network (FFN):

$W_1$  (expand):  $512 \times 2048 = 1,048,576$

$W_2$  (compress):  $2048 \times 512 = 1,048,576$

FFN total:  $\approx 2,097,152 \approx 2\text{M}$  params

One encoder layer total:  $\approx 3\text{M}$  params

### 7.2 Full Transformer (6 encoder + 6 decoder)

6 encoder layers:  $6 \times 3\text{M} = 18\text{M}$  params

6 decoder layers:  $6 \times 3\text{M} \approx 18\text{M}$  (+ cross-attention  $\approx +6\text{M}$ )  $\approx 24\text{M}$  params

Embedding table:  $\text{vocab\_size} \times 512$  (e.g.  $32000 \times 512 \approx 16\text{M}$ )

Output projection:  $\text{vocab\_size} \times 512 \approx 16\text{M}$

Full original transformer:  $\approx 40\text{M}+$  params

| Model                       | Layers      | $d_{\text{model}}$ | Heads | Approx Params      |
|-----------------------------|-------------|--------------------|-------|--------------------|
| Original Transformer (2017) | 6+6 enc+dec | 512                | 8     | $\sim 65\text{M}$  |
| GPT-2 (small)               | 12 decoder  | 768                | 12    | $\sim 117\text{M}$ |
| GPT-2 (large)               | 36 decoder  | 1280               | 20    | $\sim 774\text{M}$ |
| GPT-3                       | 96 decoder  | 12288              | 96    | $\sim 175\text{B}$ |

|             |            |      |     |       |
|-------------|------------|------|-----|-------|
| LLaMA-2 7B  | 32 decoder | 4096 | 32  | ~7B   |
| DeepSeek V3 | 61 decoder | 7168 | 128 | ~671B |

## 8. Computation Complexity

Understanding the computational cost of each component helps explain why longer context windows are expensive and why efficient attention approximations are an active research area.

| Component                  | Complexity                                | Where n and d come from                                         | Implication                                                                                                                  |
|----------------------------|-------------------------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Self-Attention             | $O(n^2 \cdot d)$                          | n = sequence length, d = model dimension                        | Quadratic in sequence length! Doubling context = 4× attention cost. This is why 1M token contexts need specialized hardware. |
| Feed Forward Network       | $O(n \cdot d \cdot d_{ff})$               | n = sequence length, $d_{ff}$ = FFN hidden dim ( $\approx 4d$ ) | Linear in sequence length. Much cheaper than attention for long sequences.                                                   |
| Full Transformer (1 layer) | $O(n^2 \cdot d + n \cdot d \cdot d_{ff})$ | —                                                               | Attention dominates for long sequences. FFN dominates for short ones.                                                        |



Why  $O(n^2)$  for attention? Every token must compute a score against every other token — n tokens × n tokens =  $n^2$  score calculations. Each calculation involves a d-dimensional dot product. Total:  $n^2 \times d$  operations. This is why ChatGPT with a 128K context window costs significantly more per token than one with 4K context.

## 9. Key Q&A from Class

| Question                                                                   | Answer                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Why not just use embedding similarity directly instead of QKV projections? | Raw embeddings are static — they represent the word's general meaning, not its role in a specific context. WQ, WK, WV projections are learned to capture task-specific relationships. The model learns to project words into spaces that are useful for computing attention, which can be very different from their general embedding space. |
| If $d_k$ is very large, what happens?                                      | Dot products become very large (many large numbers added together). Softmax saturates — one token gets ~100% and all others ~0%. Gradients vanish. That is why we scale by                                                                                                                                                                   |

|                                                                                     |                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                     | $\sqrt{d_k}$ .                                                                                                                                                                                                                                                                                                      |
| Do all words attend equally to all others? Is uniform attention a problem?          | No — Softmax ensures focused attention. Some words naturally get very high scores (e.g., 'cat' for 'sat') while others get near-zero. If somehow scores were equal, Softmax would distribute evenly, but this is rare in practice as different words have different vector content.                                 |
| Training uses parallel processing. Inference is sequential. How is training faster? | During training, all target tokens are available simultaneously (Teacher Forcing). Masking simulates sequential generation by zeroing out future positions. This allows GPU matrix operations on all positions at once. Inference must be sequential because each prediction depends on the previous output.        |
| What is the difference between 'causal' and 'masked' attention?                     | They mean the same thing. 'Masked' describes the mechanism (future positions are masked/zeroed). 'Causal' describes the property (each token only depends on previous tokens — causality in time). 'Masked self-attention' and 'causal self-attention' are interchangeable terms for the decoder's first sub-layer. |
| Why does the decoder have 3 sub-layers but the encoder only 2?                      | The decoder has three jobs: (1) understand what it has generated so far — masked self-attention, (2) understand the source input — cross-attention to encoder, (3) transform and extract patterns — FFN. The encoder only needs to understand its own input sequence (self-attention + FFN = 2 sub-layers).         |

## 10. Official Resources

| Resource                       | URL                                                                                                                                             | Description                                                        |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Lecture Slides + Materials     | <a href="https://drive.google.com/drive/.../1xR25J41uF6L98Qt64IL86-yUEJlJyumq">drive.google.com/drive/.../1xR25J41uF6L98Qt64IL86-yUEJlJyumq</a> | Full Week 3 slides + chat history on Google Drive                  |
| Attention Code (GitHub)        | <a href="https://github.com/samitmohan/ML/tree/master/algorithms/attention">github.com/samitmohan/ML/tree/master/algorithms/attention</a>       | Read-through only — attention implementation code                  |
| Implement Attention Yourself   | <a href="https://tensortonic.com/research">tensortonic.com/research</a>                                                                         | Assignment: implement full attention mechanism                     |
| LLM Visualiser                 | <a href="https://bbycroft.net/llm">bbycroft.net/llm</a>                                                                                         | Interactive 3D visualization of LLM internals — highly recommended |
| Transformer Visualiser         | <a href="https://poloclub.github.io/transformer-explainer/">poloclub.github.io/transformer-explainer/</a>                                       | Step-by-step visual walkthrough of transformer architecture        |
| Self Attention Recap (YouTube) | <a href="https://youtube.com/watch?v=4naXLhVfeho">youtube.com/watch?v=4naXLhVfeho</a> [playlist index 9, t=82s]                                 | Quick recap of self-attention mechanics — watch before next class  |
| Attention is All You           | <a href="https://youtube.com/watch?">youtube.com/watch?</a>                                                                                     | Detailed walkthrough of the                                        |

|                         |                                           |                                                                     |
|-------------------------|-------------------------------------------|---------------------------------------------------------------------|
| Need (YouTube)          | v=bCz4OMemCcA [playlist index 7, t=2275s] | original 2017 transformer paper                                     |
| Attention Paper (arXiv) | arxiv.org/abs/1706.03762                  | Original paper — read Sections 3 (Architecture) and 3.2 (Attention) |
| LayerNorm Paper (arXiv) | arxiv.org/abs/1805.11604                  | Layer Normalisation paper — referenced in class handwritten notes   |

## 11. Weekly Study Plan

| Day       | Theory Task                                                                                                                                                    | Practical Task                                                                                                                          |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Sunday    | Re-read W3D2 notes. Draw the encoder block and decoder block with all 3 sub-layers, skip connections, and normalisation positions from memory.                 | Implement skip connections in Python: $\text{output} = x + \text{layer}(x)$ . Test that when layer returns zeros, $\text{output} = x$ . |
| Monday    | Study multi-head attention. Draw the architecture: $H \text{ heads} \rightarrow \text{concatenate} \rightarrow W_O$ . Calculate parameter count for GPT-2 MHA. | Watch Self Attention Recap YouTube. Identify where MHA differs from the single-head version.                                            |
| Tuesday   | Study the three attention types. Build the $3 \times 3$ and $3 \times 4$ attention matrices for self/masked/cross attention visually.                          | Implement causal masking: create a lower-triangular matrix and set upper-triangle to $-\text{inf}$ .                                    |
| Wednesday | Study Teacher Forcing vs Autoregressive. Walk through a 5-token sequence both ways.                                                                            | Watch Attention is All You Need YouTube explanation. Follow encoder and decoder flow.                                                   |
| Thursday  | Re-read the original paper Sections 3.2-3.3. Verify skip connections, LayerNorm placement.                                                                     | Start tensortonic.com attention implementation. Add skip connections on top of what you built last week.                                |
| Friday    | Calculate parameter counts for one encoder layer from scratch. Verify against the table in Section 7.                                                          | Add LayerNorm to your implementation. Test Pre-Norm vs Post-Norm variants.                                                              |
| Saturday  | Full transformer review: draw the complete encoder-decoder with all components and explain every wire.                                                         | Complete full attention implementation on tensortonic.com/research.                                                                     |